

Document Name: Async LED Blink Controller

Document Reference: CMPE 310-ASYNC-LED-NYI-92626

Date of Publication: 9/27/2026

LEAD ENGINEER: Nyi Thu Kha, UMBC

STAKEHOLDERS:

Prof Kidd, Instructor, University of Maryland Baltimore County, Baltimore, Maryland, USA

MD Safwan Zaman, TA, University of Maryland Baltimore County, Baltimore, Maryland, USA

HIGH-LEVEL DESCRIPTION:

An embedded system based on the ELEGOO Uno R3 that allows a user to set the blink intervals of two LEDs independently through the Arduino IDE serial monitor.

DESCRIPTION:

The system uses an Arduino-compatible development board, two LEDs on digital pins 2 and 3, and a bare-metal, non-blocking software architecture. The user enters which LED to change and the desired blink interval in milliseconds. Each LED then blinks at its own independent rate. Serial communication is asynchronous and does not affect the LED timing.

RESULT SUMMARY:

The system was designed, built, and tested. It successfully accepted serial commands, updated each LED interval independently, and maintained constant blinking without using blocking delays or third-party multitasking libraries. All required tests were satisfied.

REFERENCES

1. [How to do Multitasking with the Arduino](#)
2. [The Arduino Handbook – Learn Microcontrollers for Embedded Systems](#)
3. sec04b-engineeringProcess_pt1.pdf – Engineering Process Part 1
4. sec04b-engineeringProcess_pt2.pdf – Engineering Process Part 2
5. CMPE310 asynchronous Arduino lab
6. Arduino IDE 2.3.3 or later documentation
7. [ATmega328P datasheet](#)

DEFINITIONS

The User	The person operating (not programming) the embedded system
The System	The embedded system being operated by <i>The User</i>
The Customer	The person(s) paying for the embedded system being designed and built
The Developer	The person(s) designing and building the <i>System</i>
The Evaluator	The person(s) that determine whether or not <i>The System</i> satisfies <i>The Customer-requirements</i>
The Customer Requirements	The requirements defined by <i>The Customer</i> as satisfying <i>The Contract</i>
The HL-requirements	The System's high-level technical requirements derived from <i>The Customer-requirements</i>
The Requirements	context dependent; e.g. may refer to the <i>HL-requirements</i> or <i>Customer requirements</i>
The Educational-constraints	The requirements imposed by the instructor unrelated to the embedded system that allow <i>The System</i> to be evaluated.
The Company	The organization on <i>The Customer</i> has contracted with to build <i>The System</i>
The Contract	The business document that legally binds <i>The Company</i> to provide some service or product to <i>The Customer</i>
serial-monitor	The serial port used by the Arduino IDE to communicate with <i>The User</i>
The Reference-platform	The configuration on of <i>The System</i> used by <i>The Developer</i> to test and validate <i>The System</i> . For this class, <i>The System</i> is the Arduino compatible ELEGOO Uno R3 development board
Asynchronous	Serial communication and LED timing that do not block or wait on each other
Non-Blocking	Code that does not use <code>delay()</code> to pause execution
Millis()	Arduino function returning milliseconds since program start, used for timing

Blink Interval	Full on/off period in milliseconds. On-time and off-time are each half the interval
Duty Cycle	Percentage of time the LED is on; this design used 50%

ACRONYMS AND ABBREVIATIONS

Arduino	an Italian open-source hardware and software company; also refers to a development board created by that company
arduino.h	header for a library of convenience functions specific to the Arduino development platform
AVR	A family of microcontrollers, originally developed by Atmel, and currently owned by Microchip Technology
avr.h	header including helper functions and definitions for using an AVR microcontroller
ELEGOO	A Chinese company that develops and markets 3D printers and accessories
HL	High-level
IDE	Integrated Development Environment
gcc	Front end of the GNU Compiler Collection
GitHub	A widely used distributed SVC (Software Version Control) system
LED	Light Emitting Diode
RTOS	Real-Time Operating System
ISR	Interrupt Service Routine
SVC	Software Version Control
UART	Universal Asynchronous Receiver/Trasmitter

CONVENTIONS

- Must, shall or will** - your design must satisfy the requirement
May - your design may satisfy the requirement but doesn't have to
Informative - the intent of the following description is to make the requirement more understandable

All customer requirements are started with **C.#**

All high-level technical requirements are started with **HL.#**

All testing/validation requirements are started with **T.#**

CUSTOMER REQUIREMENTS

- C1.** The User will be able to choose the blink rate of the LED with a button.
- C2.** The User will choose between 4 rates plus off.
- C3.** The blink rates must be set by an operator prior to the system being started.
- C4.** Permitted blink rates must be perceivable by the viewer.
- C5.** The minimum blink rate must be no slower than once every 5 seconds.
- C6.** The System must run upon an Arduino Uno R3 compatible board.

PART I – HIGH-LEVEL TECHNICAL REQUIREMENTS DERIVED FROM C1-C6

- HL.1** The System must use an Arduino Uno R3 compatible development board.
- HL.2** The System must use at least one LED.
- HL.3** The System must use at least one momentary push-button.
- HL.4** The System must provide five user-selectable modes: OFF four blink rates.
- HL.5** The four blink rates must be nominally 0.2 Hz, 1 Hz, 2 Hz, and 5 Hz.
- HL.6** The User must select the mode using the pushbutton before normal operation begins.
- HL.7** The selected mode must be stored and used after the System starts.
- HL.8** The slowest blink rate must have a period no greater than 5.0 seconds.
- HL.9** Each blink rate must produce alternating ON/OFF states with a 50% duty cycle.
- HL.10** The System must not require a serial monitor or host computer for mode selection.
- HL.11** The System must run standalone once programmed and powered.

Note: These Part I HLs are not used in Parts II and III.

PART II/III – HIGH-LEVEL TECHNICAL REQUIREMENTS GIVEN IN TABLE 3

Table 1. High-Level Requirements

HL.1	The System must use at least 2 LEDs.
HL.2	The System must use a standard Arduino compatible development board, like the provided ELEGOO Uno R3.
HL.3	The System must communicate with The User via the Arduino IDE serial-monitor port.
HL.4	Any use of the serial-monitor in HL.3 must be asynchronous and not affect the blinking of the LEDs.
HL.5	The User must set the blink interval of the LEDs in msec.
HL.6	The blink rate of each LED must be constant unless changed by The User.

EDUCATIONAL CONSTRAINTS

Table 2. Educational Constraints

EC.1	The Developer must use the version of gcc used by Arduino IDE 2.3.3 or later for the AVR ATmega328p to cross compile any program used by The System.
EC.2	The Developer must have any software used in The System hosted on a GitHub repository. This software does not need to include libraries, macros, or other code not developed by The Developer.
EC.2.1	The Evaluator must be able to download from the repository any software, documentation, and other material needed for their evaluation, cross compile any software, and run that software on The Reference Platform.
EC.3	LEDs must be connected to Arduino digital Pins 2 and 3.
EC.4	The Evaluator must be able to successfully have The System perform the dialog in Table 5.
EC.5	The Developer must do bare metal programming, i.e., they cannot use any 3rd party multi-tasking libraries or programs such as FreeRTOS.

DEVELOPMENT PLATFORM

1. ELEGOO Uno R3 development board or compatible Arduino Uno R3
2. Arduino IDE 2.3.3 or better
3. gcc toolchain used by Arduino IDE for AVR ATmega328p
4. $2 \times$ LEDs
5. $2 \times 200 \Omega \frac{3}{4} W$ resistors
6. $1 \times 10 k\Omega \frac{3}{4} W$ resistors
7. Breadboard and jumper wires
8. USB 2.0 female type B to female USB 2.0 type A cable
9. Laptop/computer running Arduino IDE
10. GitHub repository for source and documentation

ADDITIONAL DESIGN CONSIDERATIONS

1. LEDs are connected to digital pins 2 and 3.
2. LED current-limiting resistors are 200Ω , giving approximately 15 mA per LED, within the ATmega328p 20 mA per-pin limit.
3. The interval entered by the user is full period. On-time and off-time are each `interval / 2`.
4. Serial input is parsed without blocking.
5. `millis()` is used for all LED timing.
6. No `delay()` is used in the main loop.
7. No FREERTOS or third-party multitasking library is used.
8. The serial monitor baud rate is 9600.
9. The system remains responsive while waiting for serial input.

SCHEMATIC DESCRIPTION

- LED1: Arduino digital pin 2 $\rightarrow$ 200Ω resistor $\rightarrow$ LED1 anode; LED1 cathode $\rightarrow$ GND.
- LED2: Arduino digital pin 3 $\rightarrow$ 200Ω resistor $\rightarrow$ LED2 anode; LED2 cathode $\rightarrow$ GND.
- USB cable connects ELEGOO Uno R3 to host computer
- No button or $10 k\Omega$ resistor used.

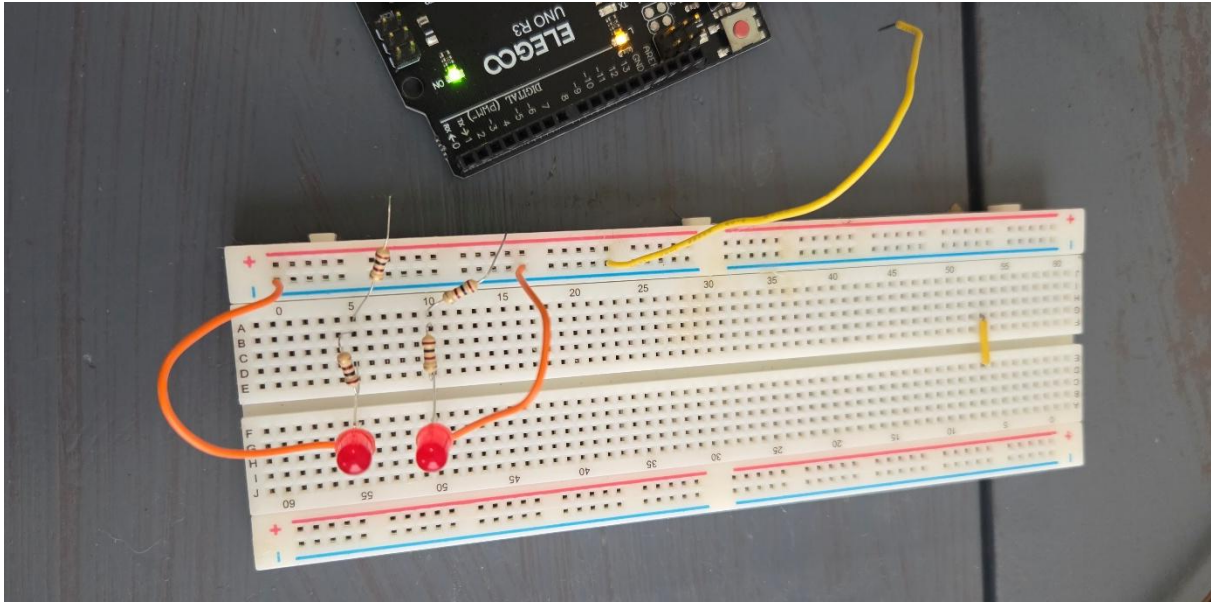


Figure 1 (Photo of physical Circuit). *Note: 2 100 Ω resistors were used for 200 Ω resistors*

TESTING AND VALIDATION REQUIREMENTS

Table 3. Testing and Validation Requirements

T.0	All testing and validation must be done on the testbed illustrated in <figure>.
T.1	Verify the System uses at least two LEDs.
T.2	Verify the System uses an Arduino-compatible development board, e.g., ELEGOO Uno R3.
T.3	Verify the System communicates with the User via the Arduino IDE serial-monitor port.
T.4	Verify serial-monitor use is asynchronous and does not affect LED blinking.
T.5	Verify the User can set each LED's blink interval in milliseconds.
T.6	Verify each LED's blink rate remains constant unless changed by the User.
T.7	Verify LEDs are connected to Arduino digital pins 2 and 3.
T.8	Verify the System performs the Table 5 dialog correctly.
T.9	Verify the source can be cross-compiled with the gcc version used by Arduino IDE 2.3.3+ for AVR ATmega328P.
T.10	Verify all developer-created software is hosted on a GitHub repository.
T.11	Verify the Evaluator can download, cross-compile, and run the software on the Reference Platform.
T.12	Verify no third-party multitasking libraries or programs such as FreeRTOS are used.
T.13	Verify bare-metal programming: direct Arduino/AVR core, <code>millis()</code> , no RTOS.
T.14	Verify LED2 set to 600 ms blinks 300 ms ON / 300 ms OFF, while LED1 is unaffected.

- T.15** Verify LED1 set to 1600 ms blinks 800 ms ON / 800 ms OFF, while LED2 is unaffected.
- T.16** Verify serial prompts: What LED? (1 or 2) and What interval (in msec)?
- T.17** Verify invalid LED numbers and non-numeric intervals are handled without crashing.

TESTING RESULTS

Table 4. Results of tests

Test performed	Results
T.0	Satisfied
T.1	Satisfied
T.2	Satisfied
T.3	Satisfied
T.4	Satisfied
T.5	Satisfied
T.6	Satisfied
T.7	Satisfied
T.8	Satisfied
T.9	Satisfied
T.10	Satisfied
T.11	Satisfied
T.12	Satisfied
T.13	Satisfied
T.14	Satisfied
T.15	Satisfied
T.16	Satisfied
T.17	Satisfied

Evidence:

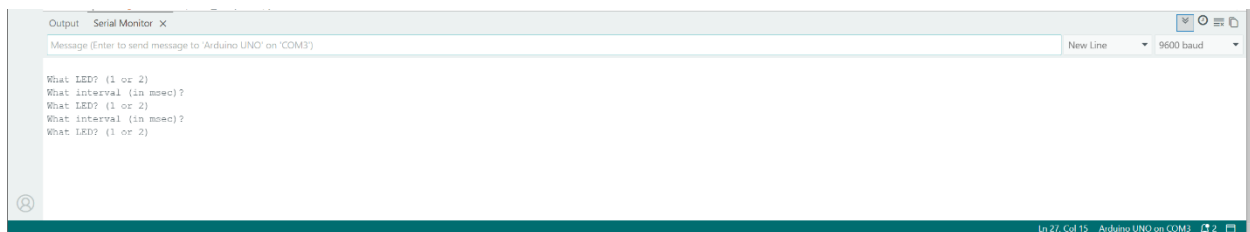


Figure 2 (Serial Monitor Screenshot): *Inputs were in this order <2, 600, 1, 1600>*

[Video of independent blinking](#)

GitHub repository link: <https://github.com/nyithukha/CMPE311/tree/main/Project1>